**UNIT-II**

**Logical data modelling**

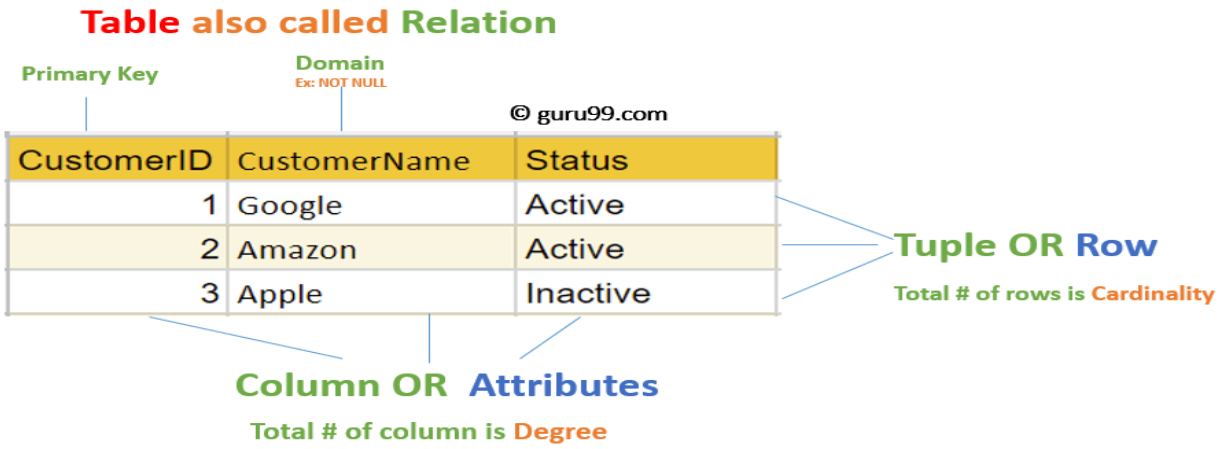
**Relational Data Model**

**Relational Model (RM)** represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables. However, the physical storage of the data is independent of the way the data are logically organized.

**Relational Model Concepts in DBMS**

1. **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME,etc.
2. **Tables** – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.
3. **Tuple** – It is nothing but a single row of a table, which contains a single record.
4. **Relation Schema:** A relation schema represents the name of the relation with its attributes.
5. **Degree:** The total number of attributes which in the relation is called the degree of the relation.
6. **Cardinality:** Total number of rows present in the Table.
7. **Column:** The column represents the set of values for a specific attribute.
8. **Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
9. **Relation key** – Every row has one, two or multiple attributes, which is called relation key.
10. **Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain




**Relational Integrity Constraints**

Relational Integrity constraints in DBMS are referred to conditions which must be present for a valid relation. These Relational constraints in DBMS are derived from the rules in the mini-world that the database represents.
There are many types of Integrity Constraints in DBMS. Constraints on the Relational database management system is mostly divided into three main categories are:

1. Domain Constraints
2. Key Constraints
3. Referential Integrity Constraints

**Domain Constraints**

Domain constraints can be violated if an attribute value is not appearing in the corresponding domain or it is not of the appropriate data type.
Domain constraints specify that within each tuple, and the value of each attribute must be unique. This is specified as data types which include standard data types integers, real numbers, characters, Booleans, variable length strings, etc.
**Example:**

```
Create DOMAIN CustomerName
CHECK (value not NULL)
```

The example shown demonstrates creating a domain constraint such that CustomerName is not NULL

**Key Constraints**

An attribute that can uniquely identify a tuple in a relation is called the key of the table. The value of the attribute for different tuples in the relation has to be unique.
**Example:**

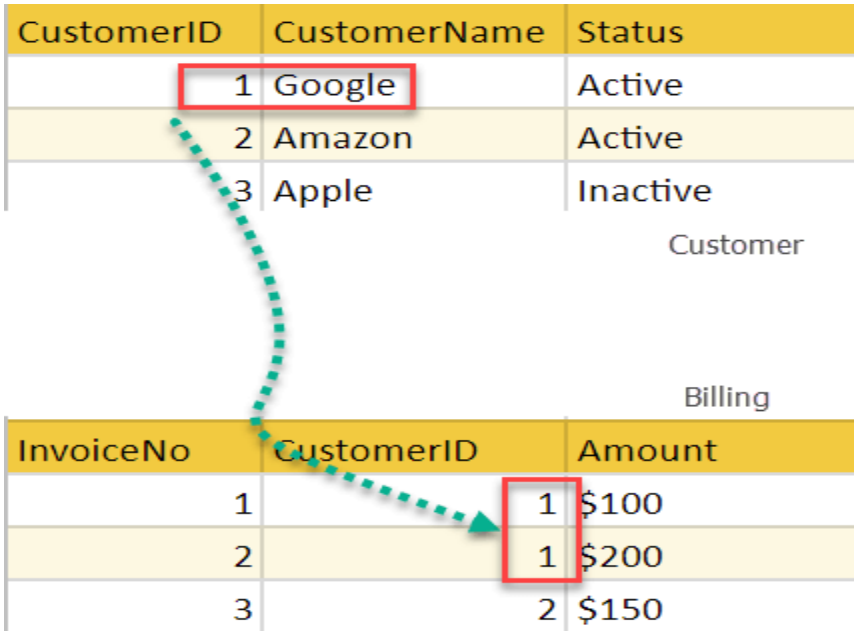
In the given table, CustomerID is a key attribute of Customer Table. It is most likely to have a single key for one customer, CustomerID =1 is only for the CustomerName =" Google".

| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 3 | Apple | Inactive |

**Referential Integrity Constraints**

Referential Integrity constraints in DBMS are based on the concept of Foreign Keys. A foreign key is an important attribute of a relation which should be referred to in other relationships. Referential integrity constraint state happens where relation refers to a key attribute of a different or same relation. However, that key element must exist in the table.

**Example:**




In the above example, we have 2 relations, Customer and Billing.

Tuple for CustomerID =1 is referenced twice in the relation Billing. So we know CustomerName=Google has billing amount $300

## Operations in Relational Model

Four basic update operations performed on relational database model are
Insert, update, delete and select.

- Insert is used to insert data into the relation
- Delete is used to delete tuples from the table.
- Modify allows you to change the values of some attributes in existing tuples.
- Select allows you to choose a specific range of data.

Whenever one of these operations are applied, integrity constraints specified on the relational database schema must never be violated.

### Insert Operation

The insert operation gives values of the attribute for a new tuple which should be inserted into a relation.

| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 3 | Apple | Inactive |


| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 3 | Apple | Inactive |
| 4 | Alibaba | Active |

### Update Operation

You can see that in the below-given relation table CustomerName= ‘Apple’ is updated from Inactive to Active.

| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 3 | Apple | Inactive |
| 4 | Alibaba | Active |


| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 3 | Apple | Active |
| 4 | Alibaba | Active |

### Delete Operation

To specify deletion, a condition on the attributes of the relation selects the tuple to be deleted.

| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 3 | Apple | Active |
| 4 | Alibaba | Active |


| CustomerID | CustomerName | Status |
|---|---|---|
| 1 | Google | Active |
| 2 | Amazon | Active |
| 4 | Alibaba | Active |

In the above-given example, CustomerName= “Apple” is deleted from the table.
The Delete operation could violate referential integrity if the tuple which is deleted is referenced by foreign keys from other tuples in the same database.

### Select Operation


In the above-given example, CustomerName=”Amazon” is selected

**Advantages of Relational Database Model**

- **Simplicity**: A Relational data model in DBMS is simpler than the hierarchical and network model.
- **Structural Independence**: The relational database is only concerned with data and not with a structure. This can improve the performance of the model.
- **Easy to use**: The Relational model in DBMS is easy as tables consisting of rows and columns are quite natural and simple to understand
- **Query capability**: It makes possible for a high-level query language like SQL to avoid complex database navigation.
- **Data independence**: The Structure of Relational database can be changed without having to change any application.
- **Scalable**: Regarding a number of records, or rows, and the number of fields, a database should be enlarged to enhance its usability.

**Disadvantages of Relational Model**

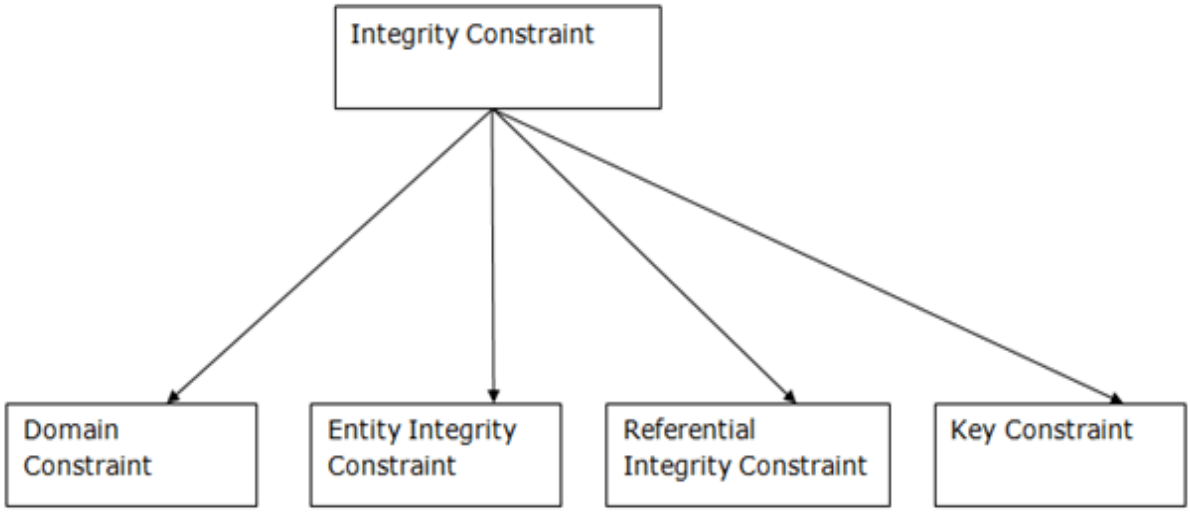
- Few relational databases have limits on field lengths which can’t be exceeded.
- Relational databases can sometimes become complex as the amount of data grows, and the relations between pieces of data become more complicated.
- Complex relational database systems may lead to isolated databases where the information cannot be shared from one system to another.

# Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.

- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint



1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

**Example:**

| ID | NAME | SEMENSTER | AGE |
|---|---|---|---|
| 1000 | Tom | 1st | 17 |
| 1001 | Johnson | 2nd | 24 |
| 1002 | Leonardo | 5th | 21 |
| 1003 | Kate | 3rd | 19 |
| 1004 | Morgan | 8th | A |

Not allowed. Because AGE is an integer attribute

2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

**Example:**

**EMPLOYEE**

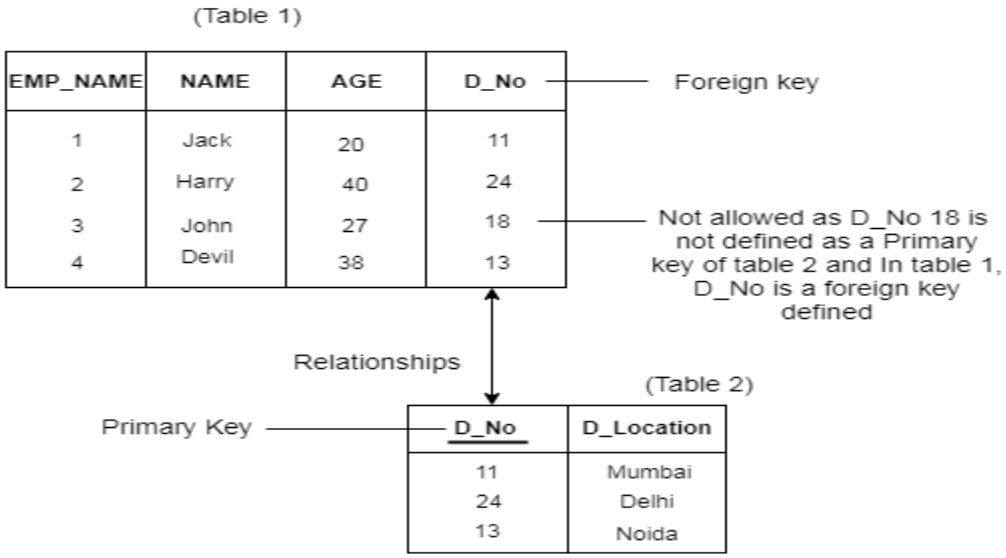
| EMP_ID | EMP_NAME | SALARY |
|---|---|---|
| 123 | Jack | 30000 |
| 142 | Harry | 60000 |
| 164 | John | 20000 |
| | Jackson | 27000 |

Not allowed as primary key can’t contain a NULL value

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

**Example:**




4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

**Example:**

| ID | NAME | SEMENSTER | AGE |
|---|---|---|---|
| 1000 | Tom | 1st | 17 |
| 1001 | Johnson | 2nd | 24 |
| 1002 | Leonardo | 5th | 21 |
| 1003 | Kate | 3rd | 19 |
| 1002 | Morgan | 8th | 22 |

Not allowed. Because all row must be unique

# Mapping ER Model to a logical schema

Entity-Relationship (E-R) diagram to a logical data model, you need to perform the following steps:

Identify entities and relationships: Start by identifying entities and relationships in the E-R diagram. An entity is a real-world object, person, or concept that can be represented in a database, while a relationship represents the connection between two entities.

Define attributes for entities: Next, define attributes for each entity that describe the characteristics of that entity. These attributes will become columns in the tables representing the entities in the logical model.

Create tables for entities: Create tables for each entity, and include columns for each of the attributes identified in the previous step.

Define relationships between tables: Define relationships between tables by adding foreign keys to the appropriate tables. A foreign key is a column in one table that refers to a primary key in another table.

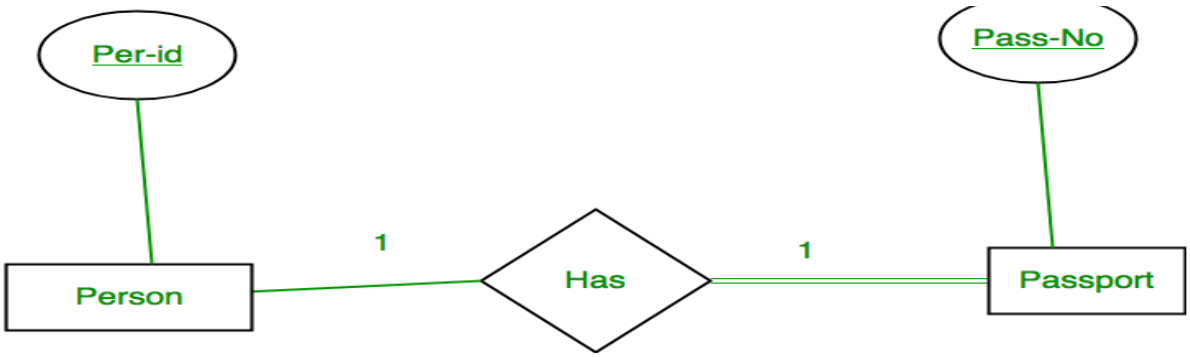
Normalize the data: Normalize the data to minimize data redundancy and improve data integrity. This process involves breaking down tables into smaller, more specialized tables to eliminate repeating data and ensure data consistency.

Add constraints: Add constraints to the data model to enforce business rules and ensure data integrity. Constraints can include primary keys, unique constraints, and referential integrity constraints.

Map the relationships: Map the relationships between tables using appropriate data relationships, such as one-to-one, one-to-many, or many-to-many relationships.

Let's understand how to map the relation with the ER model for different scenarios:

## Case 1: Binary Relationship with 1:1 cardinality with total participation of an entity



A person has 0 or 1 passport number and Passport is always owned by 1 person. So it is 1:1 cardinality with full participation constraint from Passport.

**First Convert each entity and relationship to tables.** Person table corresponds to Person Entity with key as Per-Id. Similarly the Passport table corresponds to Passport Entity with key as Pass-No. Has Table represents relationship between Person and Passport (Which person has which passport). So it will take attribute Per-Id from Person and Pass-No from Passport.

| **Person** | | | **Has** | | | **Passport** | |
|---|---|---|---|---|---|---|---|
| **Per-Id** | **Other Person Attribute** | | **Per-Id** | **Pass-No** | | **Pass-No** | **Other PassportAttribute** |
| PR1 | – | | PR1 | PS1 | | PS1 | – |
| PR2 | – | | PR2 | PS2 | | PS2 | – |
| PR3 | – | | | | | | |

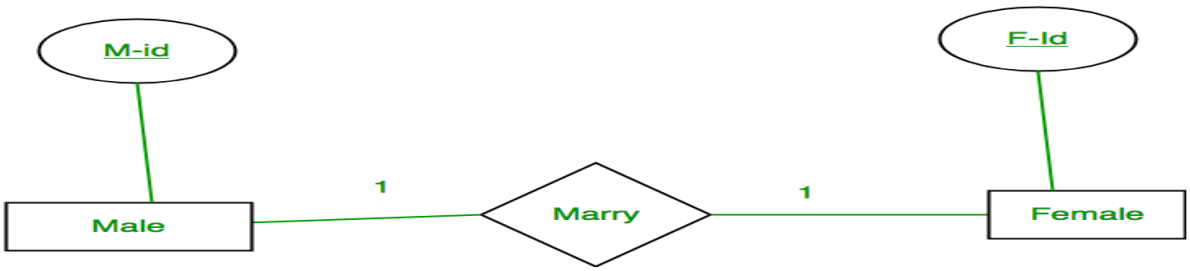
**Table 1**

As we can see from Table 1, each Per-Id and Pass-No has only one entry in Has Table. So we can merge all three tables into 1 with attributes shown in Table 2. Each Per-Id will be unique and not null. So it will be the key. Pass-No can't be key because for some person, it can be NULL.

| Per-Id | Other Person Attribute | Pass-No | Other PassportAttribute |
|---|---|---|---|

**Table 2**

## Case 2: Binary Relationship with 1:1 cardinality and partial participation of both entities




A male marries 0 or 1 female and vice versa as well. So it is 1:1 cardinality with partial participation constraint from both. First Convert each entity and relationship to tables. Male table corresponds to Male Entity with key as M-Id. Similarly Female table corresponds to Female Entity with key as F-Id. Marry Table represents relationship between Male and Female (Which Male marries which female). So it will take attribute M-Id from Male and F-Id from Female.

| Male | | | Marry | | | Female | |
|---|---|---|---|---|---|---|---|
| M-Id | Other Male Attribute | | M-Id | F-Id | | F-Id | Other FemaleAttribute |
| M1 | – | | M1 | F2 | | F1 | – |
| M2 | – | | M2 | F1 | | F2 | – |
| M3 | – | | | | | F3 | – |

**Table 3**

As we can see from Table 3, some males and some females do not marry. If we merge 3 tables into 1, for some M-Id, F-Id will be NULL. So there is no attribute which is always not NULL. So we can't merge all three tables into 1. We can convert into 2 tables. In table 4, M-Id who are married will have F-Id associated.

For others, it will be NULL. Table 5 will have information of all females. Primary Keys have been underlined.

| M-Id | Other Male Attribute | F-Id |
|---|---|---|

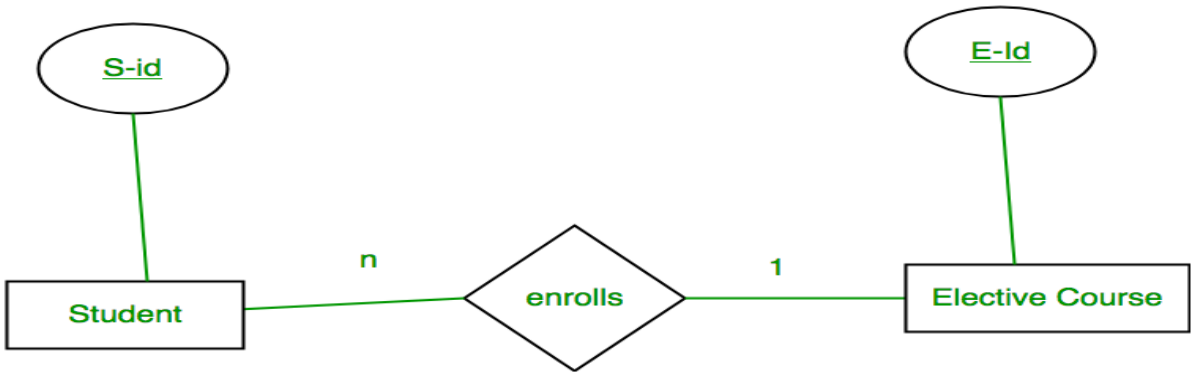
**Table 4**

| F-Id | Other FemaleAttribute |
|---|---|

**Table 5**

**Note:** Binary relationship with 1:1 cardinality will have 2 table if partial participation of both entities in the relationship. If atleast 1 entity has total participation, number of tables required will be 1.

## Case 3: Binary Relationship with n: 1 cardinality




In this scenario, every student can enroll only in one elective course but for an elective course there can be more than one student. First Convert each entity and relationship to tables. Student table corresponds to Student Entity with key as S-Id. Similarly Elective_Course table corresponds to Elective_Course Entity with key as E-Id. Enrolls Table represents relationship between Student and

Elective_Course (Which student enrolls in which course). So it will take attribute S-Id from Student and E-Id from Elective_Course.

| Student | | | Enrolls | | | Elective_Course | |
|---|---|---|---|---|---|---|---|
| S-Id | Other Student Attribute | | S-Id | E-Id | | E-Id | Other Elective CourseAttribute |
| S1 | – | | S1 | E1 | | E1 | – |
| S2 | – | | S2 | E2 | | E2 | – |
| S3 | – | | S3 | E1 | | E3 | – |
| S4 | – | | S4 | E1 | | | |

**Table 6**

As we can see from Table 6, S-Id is not repeating in Enrolls Table. So it can be considered as a key of Enrolls table. Both Student and Enrolls Table's key is same; we can merge it as a single table. The resultant tables are shown in Table 7 and Table 8. Primary Keys have been underlined.

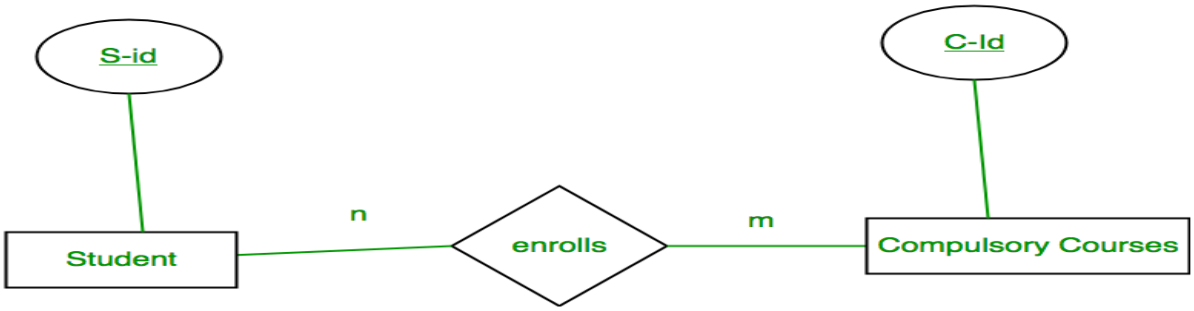
| S-Id | Other Student Attribute | E-Id |
|---|---|---|

**Table 7**

| E-Id | Other Elective CourseAttribute |
|---|---|

**Table 8**

## Case 4: Binary Relationship with m: n cardinality




In this scenario, every student can enroll in more than 1 compulsory course and for a compulsory course there can be more than 1 student. First Convert each entity and relationship to tables. Student table corresponds to Student Entity with key as S-Id. Similarly Compulsory_Courses table corresponds to Compulsory Courses Entity with key as C-Id. Enrolls Table represents relationship between Student and Compulsory_Courses (Which student enrolls in which course). So it will take attribute S-Id from Person and C-Id from Compulsory_Courses.

| Student | | | Enrolls | | | Compulsory_Courses | |
|---|---|---|---|---|---|---|---|
| S-Id | Other Student Attribute | | S-Id | C-Id | | C-Id | Other Compulsory CourseAttribute |
| S1 | – | | S1 | C1 | | C1 | – |
| S2 | – | | S1 | C2 | | C2 | – |
| S3 | – | | S3 | C1 | | C3 | – |
| S4 | – | | S4 | C3 | | C4 | – |
| | | | S4 | C2 | | | |
| | | | S3 | C3 | | | |

**Table 9**

As we can see from Table 9, S-Id and C-Id both are repeating in Enrolls Table. But its combination is unique; so it can be considered as a key of Enrolls table. All tables' keys are different, these can't be merged. Primary Keys of all tables have been underlined.

# Mapping EER Model to a logical schema

1. To draw EER diagrams first, we have to identify all entities.

- After we found entities from the scenario you should check whether those entities have sub-entities. If so you have to mark sub-entities in your diagram.
- Dividing entities into sub-entities we called as specialization. And combining sub-entities to one entity is called a generalization.

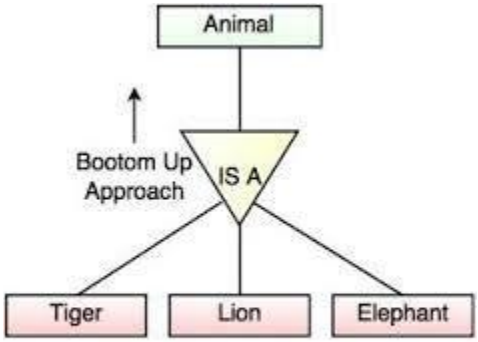



Fig. Generalization

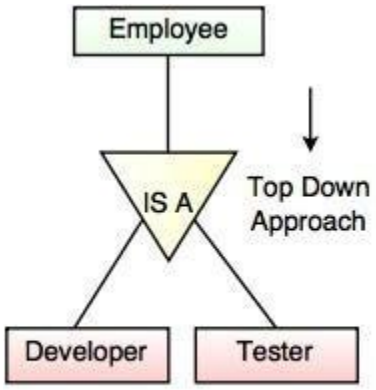



Fig. Specialization

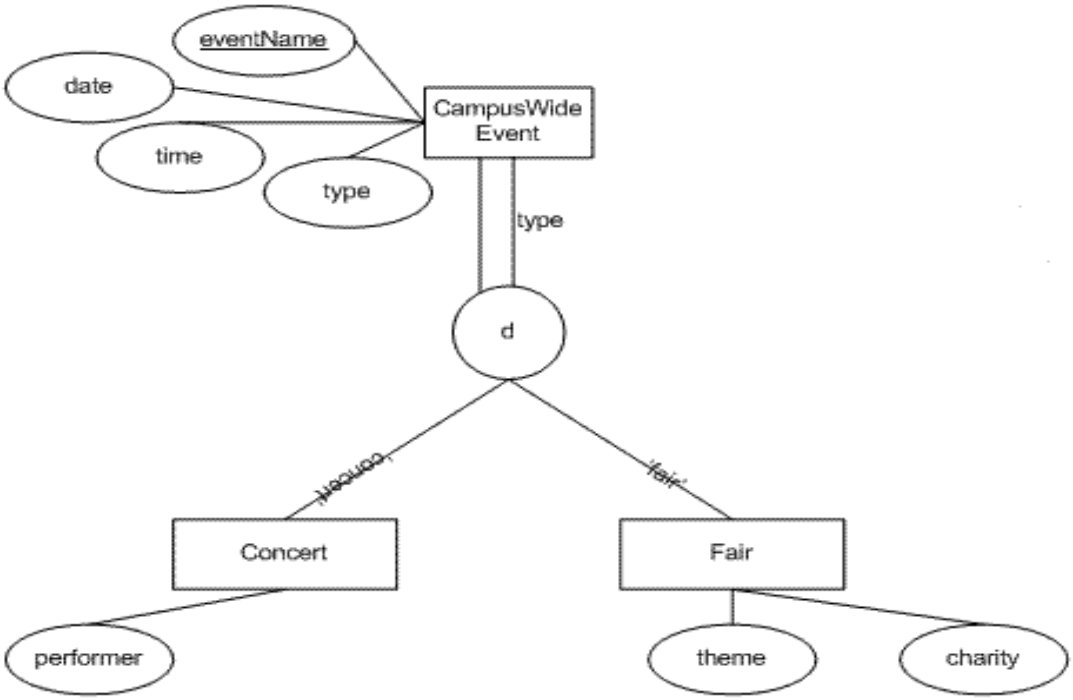
2. Then you have to identify relationships between entities and mark them.

3. Add attributes for entities. Give meaningful attribute names so they can be understood easily.

4.Mark the cardinalities and participation

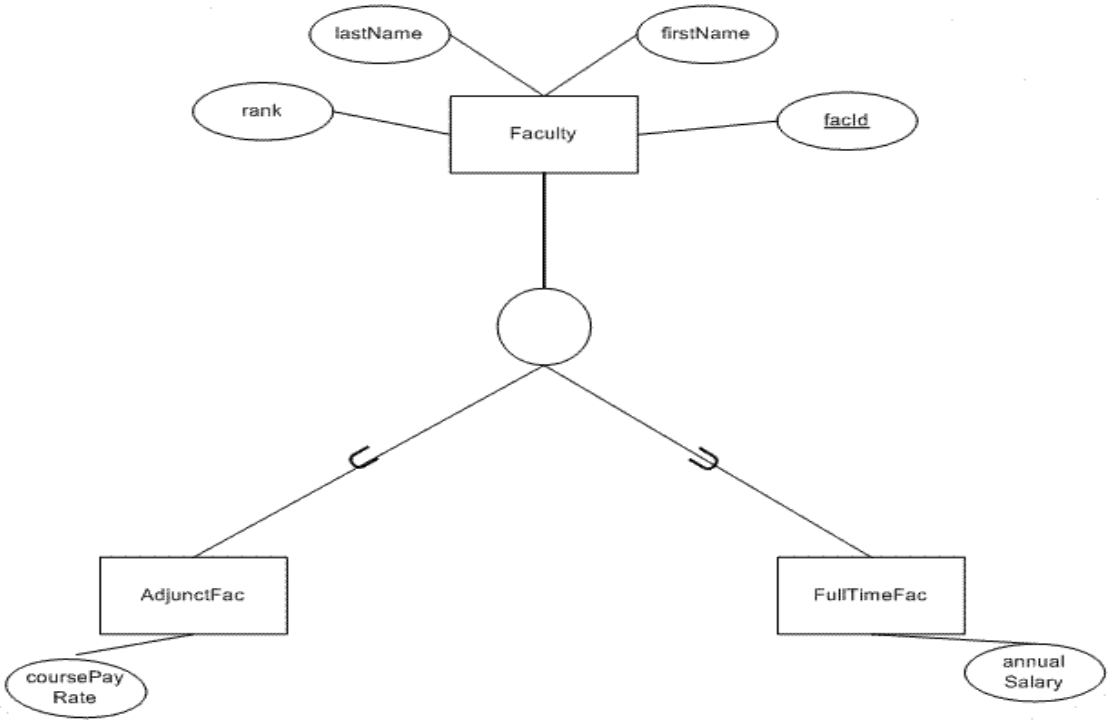
It is also required to check whether sub-entities totally depend on the main entity or not. And you should mark it.

If all members in the superclass(main entity) participate in either one subclass it is known as total participation. It is marked by double lines.




**Total Participation**

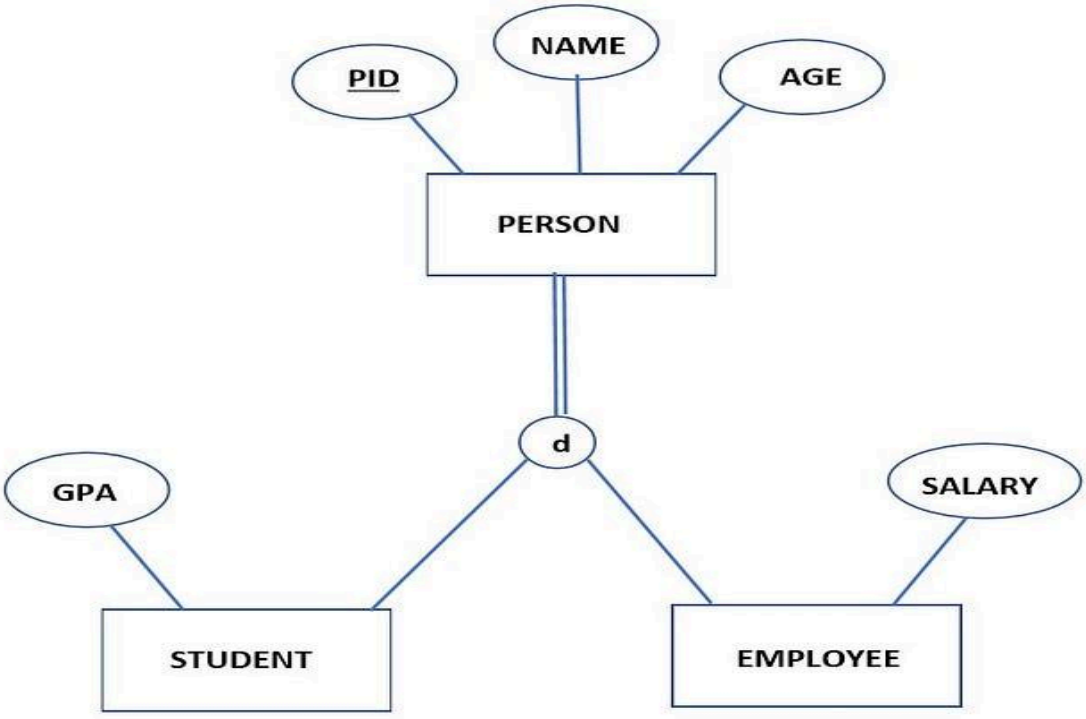
If at least one member in the superclass does not participate in subclass it is known as partial participation. It is denoted by one single line.




**Partial Participation**

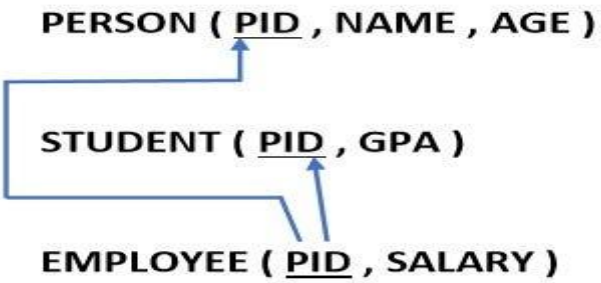
# EER diagrams into relational schemas.

Let us go through the following diagram.




There are four ways to draw relational schema for an EER. You have to choose the most suitable one. In the end, I'll give you the guidelines on how to choose the best and most suitable way.

## First method




Here we write separate relations to all superclass entities and subclass entities. And here we have to write the superclass entities' primary key to all subclass entities and then map them as shown above. Note that we write only the attributes belongs to each entity.

## Second method

STUDENT ( PID , GPA , NAME , AGE )

EMPLOYEE ( PID , SALARY , NAME , AGE )

Here we do not write the superclass entity but in each subclass entity, we write all attributes that are in superclass entity.

## Third method

PERSON ( PID , NAME , AGE , SALARY , GPA , PERSONTYPE )

Here we write only the superclass entity and write all the attributes which belong to subclass entities. Specialty in here is that to identify that PERSON is an EMPLOYEE or STUDENT we add a column as PERSONTYPE. After the table creates we can mark as a STUDENT or EMPLOYEE.

## Fourth method

PERSON ( PID , NAME , AGE , SALARY , GPA , STUDENT , EMPLOYEE )

Here instead of PERSONTYPE, we write STUDENT and EMPLOYEE both.

**Mapping of higher degree relationships**

## Degree of Relationship

In DBMS, a degree of relationship represents the number of entity types that are associated with a relationship. For example, we have two entities, one is a student and the other is a bag and they are connected with the primary key and foreign key. So, here we can see that the degree of relationship is 2 as 2 entities are associating in a relationship.

## Types of degree

Now, based on the number of linked entity types, we have 4 types of degrees of relationships.

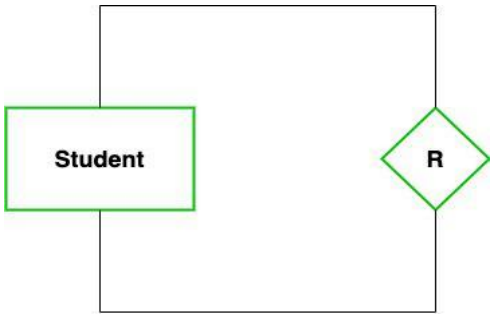
1. Unary
2. Binary
3. Ternary
4. N-ary

Let's discuss them one by one with the help of examples.

## Unary (Degree 1)

In this type of relationship, both the associating entity types are the same. So, we can say that unary relationships exist when both entity types are the same and we call them the degree of relationship is 1. In other words, in a relation only one entity set is participating then such type of relationship is known as a unary relationship.

Example: In a particular class, we have many students, there are monitors too. So, here class monitors are also students. Thus, we can say that only students are participating here. So the degree of such type of relationship is 1.

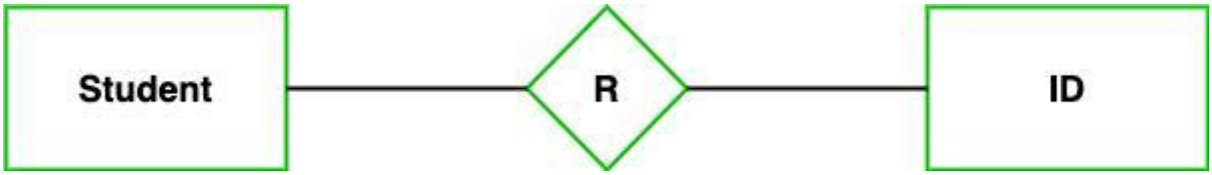



*unary*

## Binary (Degree 2)

In a Binary relationship, there are two types of entity associates. So, we can say that a Binary relationship exists when there are two types of entity and we call them a degree of relationship is 2. Or in other words, in a relation when two entity sets are participating then such type of relationship is known as a binary relationship. This is the most used relationship and one can easily be converted into a relational table.

Example: We have two entity types ‘Student’ and ‘ID’ where each ‘Student’ has his ‘ID’. So, here two entity types are associating we can say it is a binary relationship. Also, one ‘Father’ can have many ‘daughters’ but each ‘daughter’ should belong to only one ‘father. We can say that it is a one-to-many binary relationship.

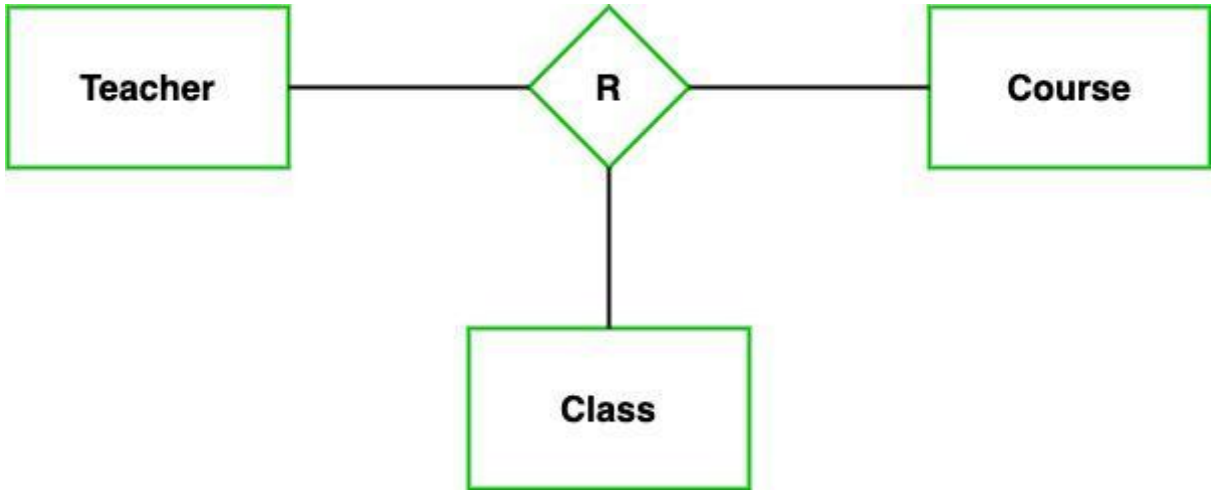



***Binary***

## Ternary (Degree 3)

In the Ternary relationship, there are three types of entity associates. So, we can say that a Ternary relationship exists when there are three types of entity and we call them a degree of relationship is 3. Since the number of entities increases due to this, it becomes very complex to turn E-R into a relational table. Now let’s understand with the examples.

Example: We have three entity types ‘Teacher’, ‘Course’, and ‘Class’. The relationship between these entities is defined as the teacher teaching a particular course, also the teacher teaches a particular class. So, here three entity types are associating we can say it is a ternary relationship.

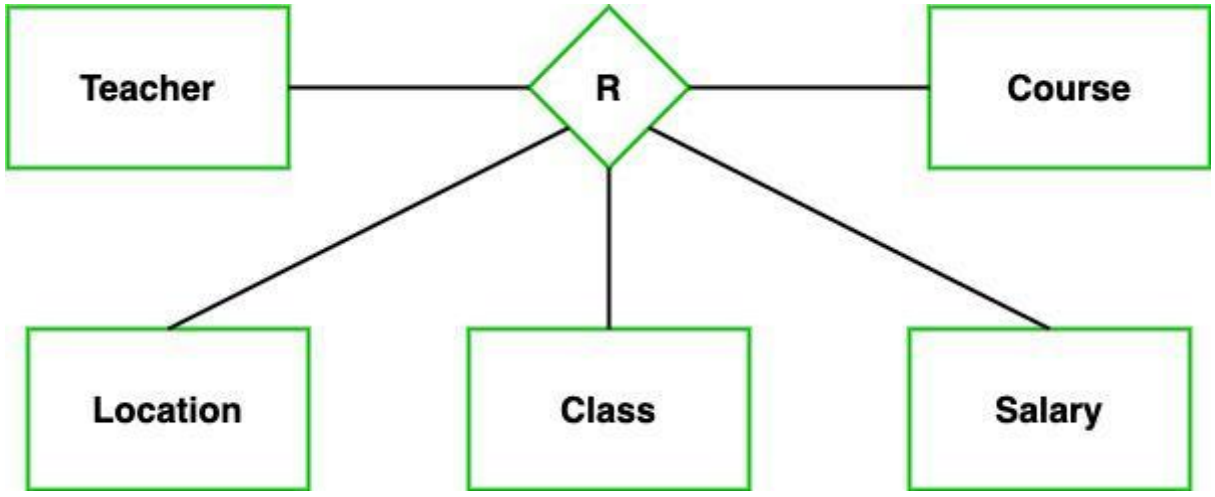



*Ternary*

## N-ary (n Degree)

In the N-ary relationship, there are n types of entity that associates. So, we can say that an N-ary relationship exists when there are n types of entities. There is one limitation of the N-ary relationship, as there are many entities so it is very hard to convert into an entity, rational table. So, this is very uncommon, unlike binary which is very much popular.

Example: We have 5 entities Teacher, Class, Location, Salary, Course. So, here five entity types are associating we can say an n-ary relationship is 5.




***N-ary***

# Relationship of higher degree

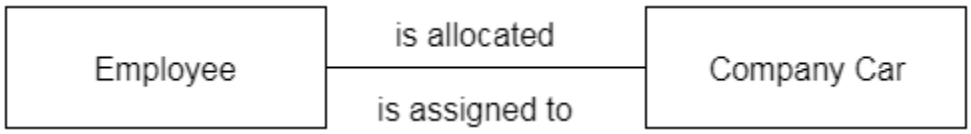
The degree of relationship can be defined as the number of occurrences in one entity that is associated with the number of occurrences in another entity.

There is the three degree of relationship:

1. One-to-one (1:1)
2. One-to-many (1:M)
3. Many-to-many (M:N)

## 1. One-to-one

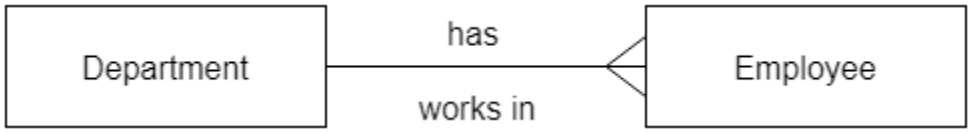
- n a one-to-one relationship, one occurrence of an entity relates to only one occurrence in another entity.
- A one-to-one relationship rarely exists in practice.
- For example: if an employee is allocated a company car then that car can only be driven by that employee.
- Therefore, employee and company car have a one-to-one relationship.




## 2. One-to-many

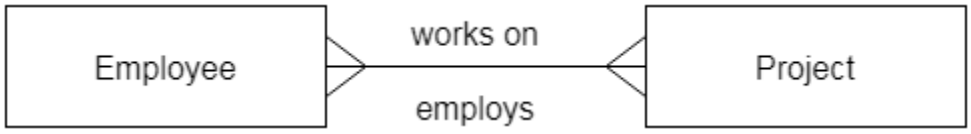
- In a one-to-many relationship, one occurrence in an entity relates to many occurrences in another entity.
- For example: An employee works in one department, but a department has many employees.

- Therefore, department and employee have a one-to-many relationship.




## 3. Many-to-many

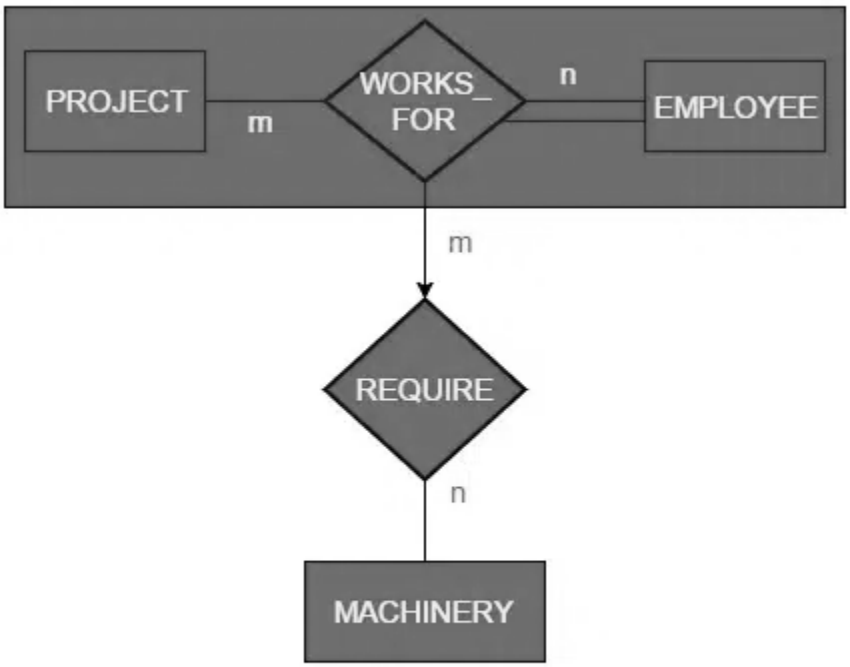
- In a many-to-many relationship, many occurrences in an entity relate to many occurrences in another entity.
- Same as a one-to-one relationship, the many-to-many relationship rarely exists in practice.
- For example: At the same time, an employee can work on several projects, and a project has a team of many employees.
- Therefore, employee and project have a many-to-many relationship.



# Aggregation

An ER diagram is not capable of representing the relationship between an entity and a relationship which may be required in some scenarios. In those cases, a relationship with its corresponding entities is aggregated into a higher-level entity. Aggregation is an abstraction through which we can represent relationships as higher-level entity sets.

For Example, an Employee working on a project may require some machinery. So, REQUIRE relationship is needed between the relationship WORKS_FOR and entity MACHINERY. Using aggregation, WORKS_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into a single entity and relationship REQUIRE is created between the aggregated entity and MACHINERY.




***Aggregation***

## Representing Aggregation Via Schema

To represent aggregation, create a schema containing the following things.

- the primary key to the aggregated relationship
- the primary key to the associated entity set
- descriptive attribute, if exists

# Normalization

A large database defined as a single relation may result in data duplication. This repetition of data may result in:

- Making relations very large.
- It isn't easy to maintain and update data as it would involve searching many records in relation.
- Wastage and poor utilization of disk space and resources.
- The likelihood of errors and inconsistencies increases.

So to handle these problems, we should analyze and decompose the relations with redundant data into smaller, simpler, and well-structured relations that are satisfy desirable properties. Normalization is a process of decomposing the relations into relations with fewer attributes.

## What is Normalization?

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

Why do we need Normalization?

The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows.

Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

Data modification anomalies can be categorized into three types:

- Insertion Anomaly: Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- Deletion Anomaly: The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- Updatation Anomaly: The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

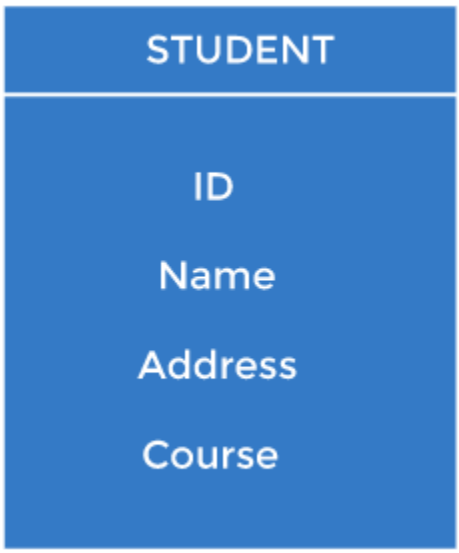
# Types of Keys in Relational Model (Candidate, Super, Primary, Alternate and Foreign)

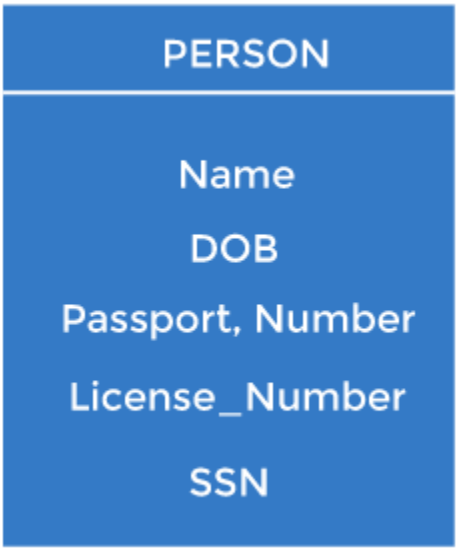
Keys are one of the basic requirements of a relational database model. It is widely used to identify the tuples(rows) uniquely in the table. We also use keys to set up relations amongst various columns and tables of a relational database.

## Different Types of Database Keys

- Candidate Key
- Primary Key
- Super Key
- Alternate Key
- Foreign Key
- Composite Key

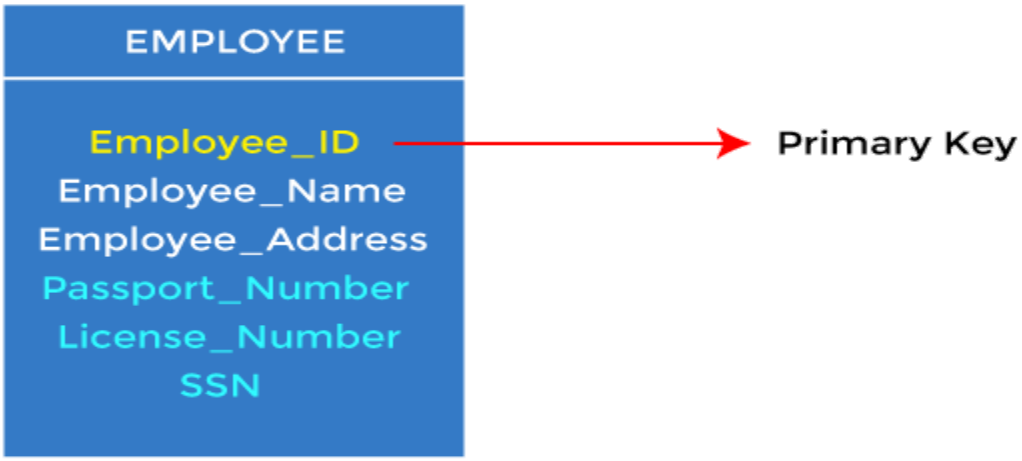
**For example,** ID is used as a key in the Student table because it is unique for each student. In the PERSON table, passport_number, license_number, SSN are keys since they are unique for each person.




## 1. Primary key

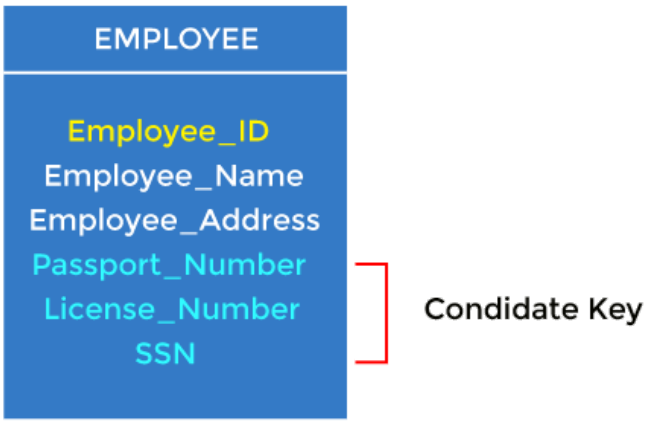
- It is the first key used to identify one and only one instance of an entity uniquely. An entity can contain multiple keys, as we saw in the PERSON table. The key which is most suitable from those lists becomes a primary key.
- In the EMPLOYEE table, ID can be the primary key since it is unique for each employee. In the EMPLOYEE table, we can even select License_Number and Passport_Number as primary keys since they are also unique.
- For each entity, the primary key selection is based on requirements and developers.




## 2. Candidate key

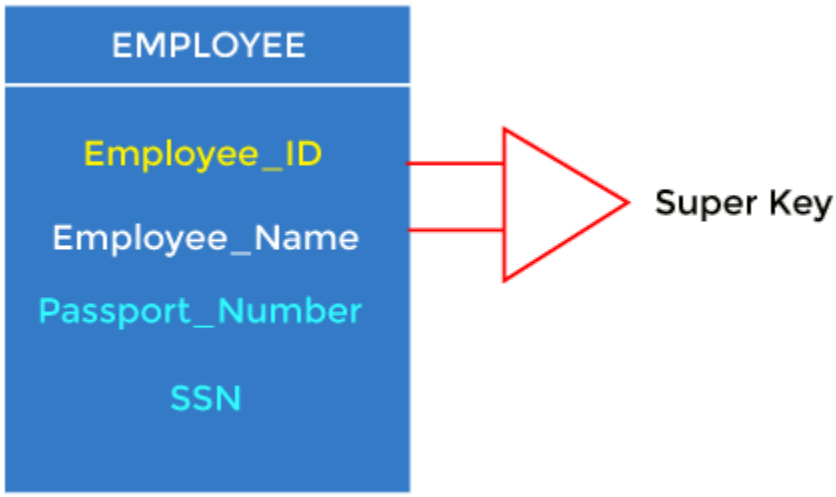
- A candidate key is an attribute or set of attributes that can uniquely identify a tuple.
- Except for the primary key, the remaining attributes are considered a candidate key. The candidate keys are as strong as the primary key.

**For example:** In the EMPLOYEE table, id is best suited for the primary key. The rest of the attributes, like SSN, Passport_Number, License_Number, etc., are considered a candidate key.



## 3. Super Key

Super key is an attribute set that can uniquely identify a tuple. A super key is a superset of a candidate key.



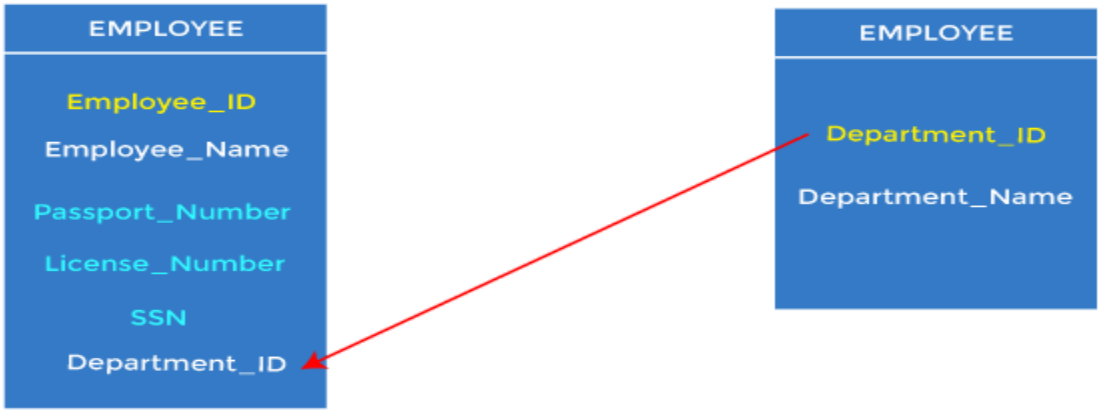

**For example:** In the above EMPLOYEE table, for(EMPLOEE_ID, EMPLOYEE_NAME), the name of two employees can be the same, but their EMPLYEE_ID can't be the same. Hence, this combination can also be a key.

The super key would be EMPLOYEE-ID (EMPLOYEE_ID, EMPLOYEE-NAME), etc.

## 4. Foreign key

- Foreign keys are the column of the table used to point to the primary key of another table.
- Every employee works in a specific department in a company, and employee and department are two different entities. So we can't store the department's information in the employee table. That's why we link these two tables through the primary key of one table.

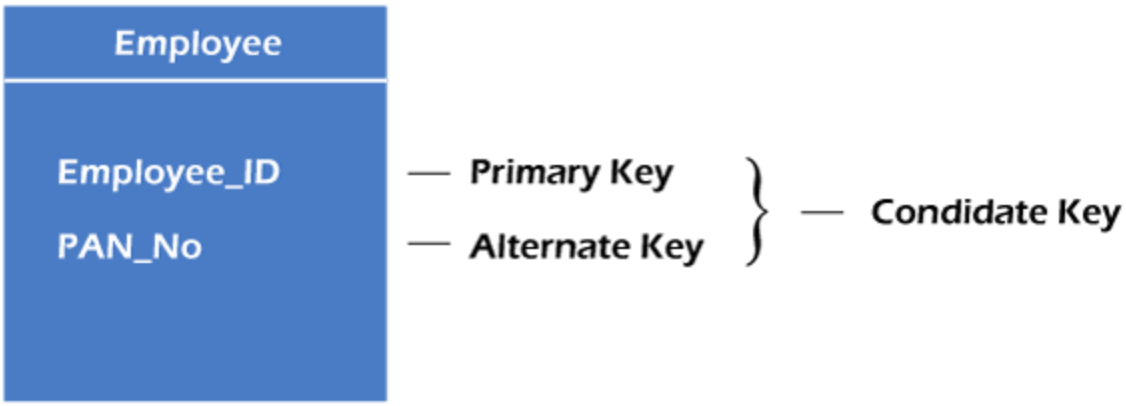
- We add the primary key of the DEPARTMENT table, Department_Id, as a new attribute in the EMPLOYEE table.
- In the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.




## 5. Alternate key

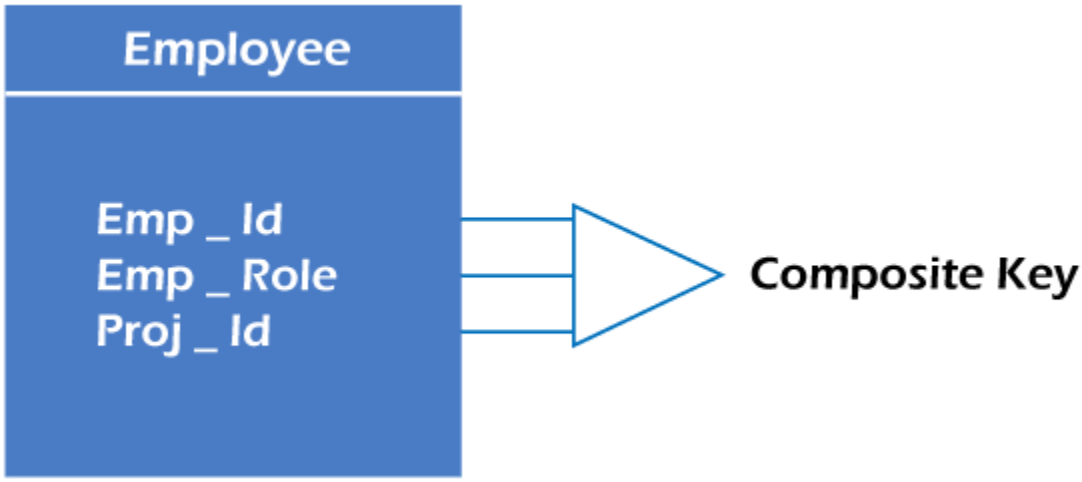
There may be one or more attributes or a combination of attributes that uniquely identify each tuple in a relation. These attributes or combinations of the attributes are called the candidate keys. One key is chosen as the primary key from these candidate keys, and the remaining candidate key, if it exists, is termed the alternate key. **In other words,** the total number of the alternate keys is the total number of candidate keys minus the primary key. The alternate key may or may not exist. If there is only one candidate key in a relation, it does not have an alternate key.

**For example,** employee relation has two attributes, Employee_Id and PAN_No, that act as candidate keys. In this relation, Employee_Id is chosen as the primary key, so the other candidate key, PAN_No, acts as the Alternate key.




## 6. Composite key

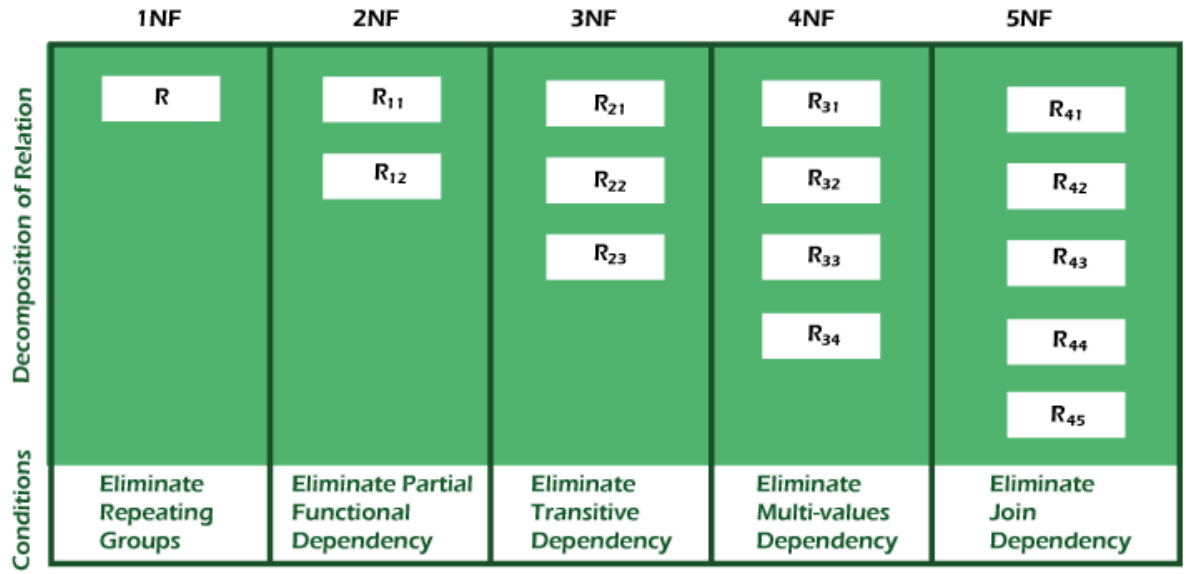
Whenever a primary key consists of more than one attribute, it is known as a composite key. This key is also known as Concatenated Key.



# Types of Normal Forms:

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.

Following are the various types of Normal forms:




| Normal Form | Description |
|---|---|
| 1NF | A relation is in 1NF if it contains an atomic value. |
| 2NF | A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key. |

| | |
|---|---|
| **3NF** | **A relation will be in 3NF if it is in 2NF and no transition dependency exists.** |
| **BCNF** | **A stronger definition of 3NF is known as Boyce Codd's normal form.** |
| **4NF** | **A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.** |
| **5NF** | **A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.** |

## First Normal Form

If a relation contain composite or multi-valued attribute, it violates first normal form or a relation is in first normal form if it does not contain any composite or multi-valued attribute. A relation is in first normal form if every attribute in that relation is **singled valued attribute**.

- **Example 1 –** Relation STUDENT in table 1 is not in 1NF because of multi-valued attribute STUD_PHONE. Its decomposition into 1NF has been shown in table 2.

| STUD_NO | STUD_NAME | STUD_PHONE | STUD_STATE | STUD_COUNTRY |
|---|---|---|---|---|
| 1 | RAM | 9716271721, 9871717178 | HARYANA | INDIA |
| 2 | RAM | 9898297281 | PUNJAB | INDIA |
| 3 | SURESH | | PUNJAB | INDIA |

**Table 1**

Conversion to first normal form

| STUD_NO | STUD_NAME | STUD_PHONE | STUD_STATE | STUD_COUNTRY |
|---|---|---|---|---|
| 1 | RAM | 9716271721 | HARYANA | INDIA |
| 1 | RAM | 9871717178 | HARYANA | INDIA |
| 2 | RAM | 9898297281 | PUNJAB | INDIA |
| 3 | SURESH | | PUNJAB | INDIA |

**Table 2**

- **Example 2 –**

```
ID   Name   Courses
-----------------
1    A      c1, c2
2    E      c3
3    M      C2, c3
```

- In the above table Course is a multi-valued attribute so it is not in 1NF.

  Below Table is in 1NF as there is no multi-valued attribute

```
ID   Name   Course
-----------------
1    A       c1
1    A       c2
2    E       c3
3    M       c2
3    M       c3
```

## Second Normal Form

To be in second normal form, a relation must be in first normal form and relation must not contain any partial dependency. A relation is in 2NF if it has **No Partial Dependency,** i.e.**,** no non-prime attribute (attributes which are not part of any candidate key) is dependent on any proper subset of any candidate key of the table. **Partial Dependency** – If the proper subset of candidate key determines non-prime attribute, it is called partial dependency.

- **Example 1** – Consider table-3 as following below.

| STUD_NO | COURSE_NO | COURSE_FEE |
|---|---|---|
| 1 | C1 | 1000 |
| 2 | C2 | 1500 |
| 1 | C4 | 2000 |
| 4 | C3 | 1000 |
| 4 | C1 | 1000 |
| 2 | C5 | 2000 |

{Note that, there are many courses having the same course fee} Here, COURSE_FEE cannot alone decide the value of COURSE_NO or STUD_NO; COURSE_FEE together with STUD_NO cannot decide the value of COURSE_NO; COURSE_FEE together with COURSE_NO cannot decide the value of STUD_NO; Hence, COURSE_FEE would be a non-prime attribute, as it does not belong to the one only candidate key {STUD_NO, COURSE_NO} ; But, COURSE_NO -> COURSE_FEE, i.e., COURSE_FEE is dependent on COURSE_NO, which is a proper subset of the candidate key. Non-prime attribute COURSE_FEE is

dependent on a proper subset of the candidate key, which is a partial dependency and so this relation is not in 2NF. To convert the above relation to 2NF, we need to split the table into two tables such as : Table 1: STUD_NO, COURSE_NO Table 2: COURSE_NO, COURSE_FEE

**Table 1**

| STUD_NO | COURSE_NO |
|---|---|
| 1 | C1 |
| 2 | C2 |
| 1 | C4 |
| 4 | C3 |
| 4 | C1 |

**Table 2**

| COURSE_NO | COURSE_FEE |
|---|---|
| C1 | 1000 |
| C2 | 1500 |
| C3 | 1000 |
| C4 | 2000 |
| C5 | 2000 |

## Third Normal Form

A relation is said to be in third normal form, if we did not have any transitive dependency for non-prime attributes. The basic condition with the Third Normal Form is that, the relation must be in Second Normal Form.

Below mentioned is the basic condition that must be hold in the non-trivial functional dependency X -> Y:

- X is a Super Key.
- Y is a Prime Attribute ( this means that element of Y is some part of Candidate Key).

## BCNF

BCNF (Boyce-Codd Normal Form) is just a advanced version of Third Normal Form. Here we have some additional rules than Third Normal Form. The basic condition for any relation to be in BCNF is that it must be in Third Normal Form.

We have to focus on some basic rules that are for BCNF:

1. Table must be in Third Normal Form.

2. In relation X->Y, X must be a superkey in a relation.

## Fourth Normal Form

Fourth Normal Form contains no non-trivial multivaued dependency except candidate key. The basic condition with Fourth Normal Form is that the relation must be in BCNF.

The basic rules are mentioned below.

1. It must be in BCNF.

2. It does not have any multi-valued dependency.

## Fifth Normal Form

Fifth Normal Form is also called as Projected Normal Form. The basic conditions of Fifth Normal Form is mentioned below.

- Relation must be in Fourth Normal Form.
- The relation must not be further non loss decomposed.

## Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

## Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.
-